

Software Requirements Specification

AI-Powered Recruitment Platform

Ahmad Hasan Cheema BSAI24006

Saad Hamid BSAI24064

1. Preface

This is the Software Requirements Specification (SRS) for an AI-powered recruitment platform. It covers what the system does, who uses it, and how it should behave. It is written for developers, testers, and anyone else involved in the project.

This is version 1.0 and covers the first release. New versions will be issued if the requirements change significantly. If you reviewed an earlier draft, note that Section 5 (System Requirements) and Section 6 (Use Cases) have been updated.

2. Introduction

a) Purpose

This SRS covers the requirements for a web-based recruitment platform that uses AI to speed up and improve the hiring process. The main goal is to cut down the time recruiters spend on manual screening and to give candidates more out of the application process.

This document covers what the system needs to do, not how it will be built. Those decisions will be handled in separate design documents.

b) Scope

The platform covers the full recruitment process from posting a job to preparing for an interview. The main features are:

- Recruiters post structured job listings
- Candidates upload resumes (PDF or DOCX) which are parsed by AI
- The system generates a Match Score and Risk Evaluation per application
- Recruiters select candidates and the system sends interview invitations automatically
- Selected candidates get access to an AI mock interview tailored to the job

The system is built with Next.js, TypeScript, PostgreSQL, the OpenAI API, and the Vercel AI SDK. It does not cover onboarding, payroll, or anything after an interview is booked. The recruiter always makes the final hiring call — the platform just helps them get there.

3. Glossary

Term	Definition
Candidate	A user who browses jobs, submits resumes, and uses the mock interview module.
Recruiter	A user who creates job listings and reviews applicant scores.
Match Score	A 0:100 rating produced by the AI showing how well a resume fits a job.
Risk Evaluation	A short AI-generated text flagging potential concerns about a candidate (e.g. skill gaps, employment gaps).
Resume Parsing	Automated extraction of structured data (skills, experience, education) from an uploaded resume file.
Mock Interview	An AI-driven practice interview session with questions based on the specific job applied to.
Next.js	The web framework used to build the frontend and API routes.
PostgreSQL	The relational database storing all system data.
OpenAI API	The external AI service used for parsing, scoring, and interview question generation.
Vercel AI SDK	Toolkit used to handle streaming AI responses in the web app.
JWT	JSON Web Token — used for user authentication.

4. User Requirements Definition

a) Functional Requirements

Recruiter

- **UR-F1** : Register, log in, and manage their account.
- **UR-F2** : Create job listings by filling in a form with job title, description, required/preferred skills, and experience level.
- **UR-F3** : View all posted listings and see how many applications each has received.
- **UR-F4** : Open an application and see the parsed resume, Match Score, and Risk Evaluation.
- **UR-F5** : Select a candidate and trigger an automatic interview invitation email.
- **UR-F6** : Close or archive a listing when it is no longer active.

Candidate

- **UR-F7** : Register, log in, and manage their account.
- **UR-F8** : Browse active job listings and read job descriptions before applying.
- **UR-F9** : Apply to a job by uploading a resume in PDF or DOCX format.
- **UR-F10** : See their Match Score after applying.
- **UR-F11** : Access the AI Mock Interview module once they have been selected.
- **UR-F12** : Receive an email notification when selected for an interview.

b) Non-Functional Requirements

- **UR-NF1** : The interface should be straightforward a recruiter with no technical background should be able to use it without training.
- **UR-NF2** : The system should respond quickly. Pages and scores should load within a few seconds at most.
- **UR-NF3** : Resumes and personal data must only be visible to authorised users.
- **UR-NF4** : The system must be reliable data should be saved correctly and the platform should be available during working hours.

- **UR-NF5** : The system should work on any modern browser on desktop or laptop.

5. System Requirements Definition

a) Functional Requirements

Authentication

- **SR-F1** : The system shall support two roles: Recruiter and Candidate, each with separate access controls.
- **SR-F2** : Authentication shall use JWTs expiring after 24 hours of inactivity.
- **SR-F3** : Email format and minimum password length (8 characters) shall be validated at registration.

Job Listings

- **SR-F4** : Job listings shall include: title, department, location, employment type, required skills, preferred skills, experience level, and description. All required fields are validated before saving.
- **SR-F5** : Listings shall be stored in PostgreSQL with a creation timestamp, recruiter ID, and a status field (draft / active / closed).
- **SR-F6** : A public API endpoint shall return all active listings for candidate browsing.

Resume Parsing

- **SR-F7** : The system shall accept PDF, DOCX and TXT uploads only, up to 5 MB. Files outside these constraints shall be rejected with a clear error.
- **SR-F8** : On upload, resume text is sent to the OpenAI API with a prompt to extract: name, contact info, work experience, education, skills, and certifications. The output is stored as structured JSON linked to the application.
- **SR-F9** : If parsing fails after two retries, the application is flagged for manual review and the recruiter is notified.

Match Scoring and Risk Evaluation

- **SR-F10** : The system shall generate a Match Score (0:100) by sending the parsed resume and job requirements to the OpenAI API. The prompt evaluates skill match, experience, education, and keyword alignment.
- **SR-F11** : A Risk Evaluation shall also be generated as a short text output noting concerns such as skill gaps, employment gaps, or overqualification.
- **SR-F12** : Both are stored in the application record and shown to the recruiter. If scoring fails after two retries, 'Score unavailable' is displayed.

Interview Scheduling

- **SR-F13** : When a recruiter selects a candidate, the system shall send an email containing: job title, recruiter name, proposed time, and a link to the mock interview module.
- **SR-F14** : The send timestamp is recorded in the application record and status is updated to 'interview scheduled'. On email failure, the recruiter is notified via the dashboard.

Mock Interview

- **SR-F15** : The module is only accessible to candidates with status 'interview scheduled'.
- **SR-F16** : The system shall generate 5:10 interview questions (behavioural, technical, situational) from the job description via the OpenAI API.
- **SR-F17** : After each answer is submitted, the system sends the question and answer to the API and returns streamed feedback covering: relevance, depth, and improvement suggestions.
- **SR-F18** : Sessions are saved so candidates can return to review them. Multiple sessions per application are supported.

c) Recruiter Tools

- **Job Description Optimiser** — Before posting, AI reviews the listing and suggests edits to attract better candidates.
- **Duplicate Application Detector** — Flag candidates who applied to multiple roles so recruiters are not reviewing them twice.
- **Predictive Hiring Timeline** — Estimate how long it will take to fill a role based on the applicant pool.

b) Non-Functional Requirements

- **SR-NF1** : Non-AI API responses shall complete in under 500 ms under normal load. AI operations (parsing, scoring) shall complete within 15 seconds with a loading indicator shown.
- **SR-NF2** : The system shall support at least 200 concurrent users without degraded response times.
- **SR-NF3** : All traffic shall use HTTPS. API routes shall validate JWTs on every request and return 401 for unauthorised access.
- **SR-NF4** : Resume files shall be stored in a private cloud storage bucket — not publicly accessible by URL.
- **SR-NF5** : Target uptime is 99.5%, excluding scheduled maintenance. All DB writes shall use transactions. AI API calls shall be retried up to twice on failure.
- **SR-NF6** : AI prompts shall be stored as external configuration files, not hardcoded, so they can be updated without code changes.

6. System Models

a) Use Case Models

The tables below cover the five main use cases. The main actors are Recruiter and Candidate. The OpenAI API and Email Service are also involved as supporting actors.

UC-01: Post a Job Listing	
Use Case ID	UC-01
Actors	Recruiter
Description	A recruiter creates and publishes a new job listing on the platform.
Preconditions	The recruiter is logged in and wants to post a new role.
Main Flow	The recruiter hits 'Create New Listing' and fills in the job form. Once they submit, the system checks the input and, if everything looks good, saves the listing as active and shows the published page.
Alternate Flows	If any required fields are missing, the form highlights them and won't submit. If something goes wrong saving to the database, the recruiter sees an error and their input is kept so they don't lose anything.
Postconditions	The listing is live and candidates can see it.

UC-02: Apply and Receive Match Score

Use Case ID	UC-02
Actors	Candidate, OpenAI API
Description	A candidate uploads their resume and the system generates a Match Score and Risk Evaluation against the job.
Preconditions	The candidate is logged in and viewing a job they want to apply for.
Main Flow	The candidate clicks ‘Apply Now’ and uploads their resume (PDF or DOCX, up to 5 MB). The system checks the file, stores it, and kicks off the AI processing — first parsing the resume, then scoring it against the job requirements. Once that’s done, the candidate can see their Match Score and Risk Evaluation from their dashboard.
Alternate Flows	If the file is the wrong type or too large, the upload is rejected with a clear message. If parsing fails after two attempts, the application is flagged for manual review. If scoring fails after two attempts, the score shows as unavailable.
Postconditions	The application is saved with the parsed resume, Match Score, and Risk Evaluation all visible to the recruiter.

UC-03: Review Applications and Select a Candidate

Use Case ID	UC-03
Actors	Recruiter
Description	A recruiter reviews applications for a job listing and selects a candidate for interview.
Preconditions	The recruiter is logged in and has at least one active listing with applications on it.
Main Flow	The recruiter opens a listing and sees all the applications in a sortable table. They sort by Match Score, open the one they like, and review the parsed resume, score, and risk flags. If they want to move someone forward, they click ‘Select for Interview’ and confirm — the system updates the status and sends the invite automatically.
Alternate Flows	If there are no applications yet, the system shows a notice instead. If the recruiter changes their mind on the confirmation, nothing changes.
Postconditions	The application is marked as ‘interview scheduled’ and the email invite flow kicks off.

UC-04: Send Interview Invitation Email

Use Case ID	UC-04
Actors	System (triggered by UC-03), Email Service, Candidate
Description	The system automatically emails the candidate with interview details and a link to the mock interview module.
Preconditions	A recruiter has just selected a candidate in UC-03 and the candidate has a valid email address on file.
Main Flow	The system pulls together the candidate's email, the recruiter's name, and the job details, then fires off an email with the interview time and a link to the mock interview module. The send time gets logged against the application.
Alternate Flows	If the email service fails, the system tries once more after 60 seconds. If it fails again, the recruiter gets an alert on their dashboard. If the email address is invalid, the send is skipped and the recruiter is notified.
Postconditions	The candidate has their invite and the mock interview module is unlocked for them.

UC-05: Conduct AI Mock Interview

Use Case ID	UC-05
Actors	Candidate, OpenAI API
Description	A selected candidate completes a practice interview session. Questions are generated from the job description and feedback is given on each answer.
Preconditions	The candidate is logged in and their application is marked as 'interview scheduled'.
Main Flow	The candidate opens the mock interview module and the system generates between 5 and 10 questions based on the job description. Questions come up one at a time — the candidate types their answer and gets streamed feedback covering how relevant and detailed their response was, plus suggestions. This repeats for each question, and at the end they get a full session summary they can come back to later.
Alternate Flows	If question generation fails after two tries, the system shows an error and asks the candidate to try again later. If feedback fails for a single question, that question just shows 'Feedback unavailable' and the candidate can keep going. If they try to submit an empty answer, the system nudges them to write something first.

Postconditions

The full session is saved — every question, answer, and piece of feedback — and the candidate has had real practice tailored to the actual job.

7. Requirements Elicitation

a) Elicitation Technique

Two methods were used: Brainstorming and Analysis of Existing Systems. As a student project without access to real recruiters, these were the most realistic ways to figure out what the platform needed to do.

Brainstorming helped the team work out what the platform should do. The team started with the basic goal of reducing manual screening work, then went through possible features and narrowed them down to the ones that made sense. It worked well for an AI project because it let the team think freely about new ideas, like AI-generated risk flags and mock interview feedback, without just copying what other tools already do.

We also looked at platforms like LinkedIn and Workable to see how recruitment tools currently work. Most of them only do basic keyword matching and give candidates no feedback at all. Seeing that made it clear that adding AI scoring and a mock interview feature would be useful, and it also helped with smaller decisions like which file types to support and how to lay out a job listing.

b) Stakeholder Identification

The two main users are Recruiters and Candidates. Recruiters deal with a lot of applications and reading through them all manually takes time and can be inconsistent. The AI helps by scoring each application and flagging any concerns automatically, so recruiters can focus on the strongest candidates. Candidates usually hear nothing back after applying, with no idea how well they did. The platform shows them their Match Score after applying, and if they get selected, gives them a mock interview with feedback so they can actually prepare.